

THE FUJINET™ WIFI NETWORK PERIPHERAL

FUJINET

```
10 OPEN #1,4,0,  
    "N:HTTP://FUJINET.ONLINE/"  
20 STATUS #1,5  
30 BW=PEEK(746)+PEEK(747)*256  
40 IF BW=0 THEN 20  
50 INPUT #1,A$:PRINT A$  
60 GOTO 20  
  
70 REM THE WORLD, ONE OPEN  
    AT A TIME.
```

PROGRAMMER'S GUIDE

A TECHNICAL REFERENCE FOR THE ATARI® 8-BIT COMPUTERS

CONTENTS

TWO ROADS TO THE FUJINET	2
---------------------------------	----------

GETTING THE N: HANDLER	3
-------------------------------	----------

THE N: DEVICE FROM BASIC	4
---------------------------------	----------

THE XIO COMMANDS (NDEV)	6
--------------------------------	----------

TALKING TO SIO DIRECTLY	9
--------------------------------	----------

THE NETWORK DEVICE – SIO REFERENCE	11
---	-----------

THE FUJI CONTROL DEVICE – SIO REFERENCE	14
--	-----------

TELLING TIME	18
---------------------	-----------

WHAT NDEV CANNOT REACH	19
-------------------------------	-----------

OTHER LANGUAGES THROUGH CIO	20
------------------------------------	-----------

FUJINET FROM ATARI LOGO (NDEV)	21
---------------------------------------	-----------

FUJINET IN ACTION! (SIO)	22
---------------------------------	-----------

APPENDIX A – ERROR CODES	28
---------------------------------	-----------

APPENDIX B – COMMAND QUICK REFERENCE	29
---	-----------

APPENDIX C – THE NDEV HANDLER	31
--------------------------------------	-----------

APPENDIX D – NETCAT	42
----------------------------	-----------

TWO ROADS TO THE FUJINET

The FujiNet is, electrically, a little computer of its own — an ESP32 with WiFi, a memory-card slot, and a cable that pretends to be an Atari peripheral. It plugs into the SIO (serial) port like a disk drive, and inside it lives a cluster of **SIO devices**: eight disk drives, a printer, a modem, a clock — and the two devices this book is about, the **Network device** and the **Fuji control device**.

There are two heights at which an Atari program can reach them, and this guide teaches both, side by side.

THE HIGH ROAD — THE N: HANDLER (NDEV)

NDEV is a CIO device handler, letter **N**, that you load into memory once. From then on the network is just another Atari device: you **OPEN #1,4,0,"N:HTTP://..."**, **INPUT** and **PRINT** and **GET** and **PUT**, and **CLOSE**, exactly as you would to **D:** or **E:** or the printer. Any language that speaks CIO speaks to the FujiNet — Atari BASIC, the Assembler/Editor cartridge, Atari Logo. **All of the BASIC examples in this book take the high road.**

THE LOW ROAD — THE SIO BUS ITSELF

Underneath NDEV, and underneath CIO, is the Serial Input/Output bus. You fill a **Device Control Block** at **\$0300**, call the OS **SI0V** vector at **\$E459**, and the command travels the wire to the FujiNet. This is how you reach the **whole** FujiNet — including the Fuji control device, which NDEV does not expose at all. **All of the assembly examples in this book take the low road.**

WHEN TO USE WHICH

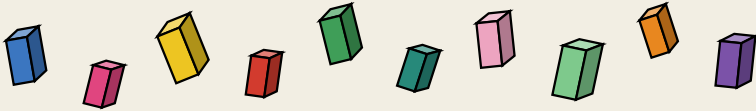
Take the high road for ordinary network I/O from a high-level language: fetching a URL, reading a socket, writing a file. Take the low road when you need speed, when NDEV is not present (or cannot be — see the next chapter), or when you must reach a command NDEV cannot: mounting disks, scanning WiFi, reading the clock, the app-key store. A real program often takes both — NDEV for the socket, direct SIO for the mount.

THE DEVICES ON THE BUS

Each FujiNet function answers to its own **device id** on the SIO bus. You will meet three:

Device	Bus id	Reached by	What it is
Network	\$71–\$78	NDEV or SIO	N1: through N8: — TCP, UDP, HTTP, TNFS, FTP, SMB, SSH, TELNET
Fuji control	\$70	SIO only	the device CONFIG talks to: mounts, hosts, slots, WiFi, clock, app keys
Clock (APETime)	\$45	SIO only	time of day in several formats

The Network device is really **eight** devices, one per channel: **N1:** is bus id **\$71**, **N2:** is **\$72**, and so on up to **N8:** at **\$78**. The Atari OS builds the on-wire id for you from the two DCB bytes **DDEVIC** and **DUNIT** — it sends **DDEVIC + DUNIT - 1** — so you set **DDEVIC** to **\$71** and **DUNIT** to the channel number, and the OS does the arithmetic. NDEV does exactly this.



GETTING THE N: HANDLER

The N: handler is not built into the FujiNet firmware or the Atari ROM — it is a small relocatable driver, **NDEV**, that you load into your Atari's memory. It ships as an executable named **AUTORUN.SYS** (or **NDEV.COM**), about a kilobyte long, that installs the **N:** device into the handler table (HATABS) and steps out of the way. Once loaded it serves **N1:** through **N8:** until the next cold start.

THREE PLACES TO GET IT

1 · THE READY-MADE HANDLER DISK

The simplest source is the handler disk image, kept on the public server **apps.irata.online**:

https://apps.irata.online/Atari_8-bit/n-handler.atr

apps.irata.online is directly reachable from a FujiNet **both** over HTTPS and over TNFS — add it as a host slot in CONFIG (just the bare name **apps.irata.online**) and you can mount **n-handler.atr** straight off it, no download to a PC required.

2 · ANY OF THE DOS DISKS

Every DOS image in the collection at

https://apps.irata.online/Atari_8-bit/DOS/

already carries the handler as its **AUTORUN.SYS**, so booting any one of them installs **N:** automatically. This is the usual way: boot a FujiNet-aware DOS, and the network is simply there.

3 · BUILD IT FROM SOURCE

The handler is open source. Its repository is

<https://github.com/FujiNetWIFI/fujinet-nhandler>

with the Atari handler under the **handler/** directory. It is written for the MADS assembler and post-processed into a **relocatable** executable (see the repository's **README** for the RELGEN dance, and Appendix C for the complete listing). Building it yourself is the way to change how it behaves — a longer receive buffer, a different interrupt policy, or one of the missing commands catalogued in Chapter 9.

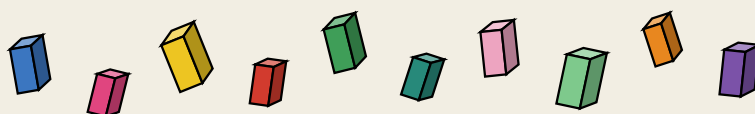
LOADING IT

NDEV loads like any DOS executable. Booted as **AUTORUN.SYS** it installs at power-on and prints a one-line FujiNet banner. Loaded by hand from DOS (the **L** “binary load” menu item, or from BASIC-adjacent loaders) it installs the moment it finishes loading. Because it relocates itself to the top of free memory and lifts **MEMLO** above itself, it coexists with whatever you load next.

IMPORTANT — DUP AND ATARI DOS 2

NDEV **cannot be used from the DUP menu of Atari DOS 2.0S, 2.0D, or 2.5**. When you return to DOS and its menu processor (DUP.SYS) loads, DUP inevitably overwrites the very region of memory NDEV relocated itself into — because NDEV lives just above **MEMLO**, and DUP claims that same space for its buffers. The handler is clobbered, and **N:** stops answering.

This is a limitation of where DOS 2's DUP lives, not of the handler. Use **N:** from a running program (BASIC, an assembled binary, a language cartridge) rather than from the DOS menu; or use a DOS whose command processor leaves the handler alone (SpartaDOS, MyDOS, and others reload cleanly). From **within** a program the handler is solid — it is only the DUP menu itself that steps on it.



THE N: DEVICE FROM BASIC

With NDEV loaded, the network is an Atari device. Everything you already know about `OPEN`, `CLOSE`, `INPUT`, `PRINT`, `GET`, `PUT` and `STATUS` applies unchanged. This chapter is the whole of ordinary network I/O from Atari BASIC.

THE DEVICE SPEC

What you open is a **device spec**: the device name, then a scheme, then the resource.

`Nx:SCHEME://host[:port]/path`

`x` is the channel, 1 through 8 (a bare `N:` means `N1:`). The scheme is one of `TCP`, `UDP`, `HTTP`, `HTTPS`, `TNFS`, `FTP`, `SMB`, `SSH`, `TELNET`, in upper case. The rest names the host and resource, e.g. `N:HTTP://fujinet.online/` or

`N1:TCP://192.168.1.5:9000/`.

OPEN — MODE AND TRANSLATION

`OPEN #ch,aux1,aux2,"Nx:..."` hands NDEV two bytes. **aux1** is the access mode; **aux2** is the end-of-line translation.

aux1	Mode
4	read (GET)
6	directory read
8	write (PUT)
9	append
12	read / write
13	HTTP POST
5	HTTP DELETE

aux2	Line endings
0	none — binary
1	CR
2	LF
3	CR / LF

Translation converts between the Atari's end-of-line (`$9B`, EOL) and the wider world's CR/LF as bytes cross the wire — indispensable for text protocols, switched off (`0`) for binary. For most reading and writing of Internet text, open with translation `2` (LF).

LISTING 1. FETCH A URL AND PRINT IT

```
10 REM --- FETCH A WEB PAGE OVER N: ---
20 DIM L$(255)
30 OPEN #1,4,2,"N:HTTP://fujinet.online/"
40 REM AUX1=4 READ, AUX2=2 LF TRANSLATION
50 STATUS #1,S
60 IF S<>1 THEN GOTO 200
70 BW=PEEK(746)+PEEK(747)*256:REM BYTES WAITING
80 IF BW=0 THEN GOTO 50
90 INPUT #1,L$
100 PRINT L$
110 GOTO 50
200 REM S=136 IS END OF FILE, THE NORMAL FINISH
210 CLOSE #1
220 END
```

READING — INPUT, GET, AND STATUS

`INPUT #ch,A$` reads one **record** — bytes up to and including an EOL. `GET #ch,B` reads a single byte into a numeric variable. NDEV keeps a receive buffer that fills as data arrives from the network; a read drains it.

Before you read, ask how much is waiting. `STATUS #ch,S` runs the handler's status entry, which asks the FujiNet and fills the four-byte OS status buffer **DVSTAT** at `$02EA` (decimal 746):

PEEK	Address	Meaning
<code>PEEK(746)+PEEK(747)*256</code>	<code>\$02EA/EB</code>	bytes waiting to be read
<code>PEEK(748)</code>	<code>\$02EC</code>	connection: 1 = up, 0 = closed by far end
<code>PEEK(749)</code>	<code>\$02ED</code>	device error (1 = OK, 136 = end of file)

The value **S** that **STATUS** returns is the handler status; when it is **136** the resource is fully read (end of file) and it is time to **CLOSE**. Watch bytes-waiting to know when a socket has data, and the connection byte to know when the far end has hung up.

THE PROCEED INTERRUPT

NDEV does not busy-wait for the network. It arms the SIO **PROCEED** interrupt (vector **VPRCED** at **\$0202**); the FujiNet pulses the line when data arrives or the connection changes, and the handler notes it. From BASIC you need not touch any of this — **GET** and **STATUS** ride on it — but it is why a **GET** on an idle socket blocks politely instead of spinning.

WRITING — PRINT AND PUT

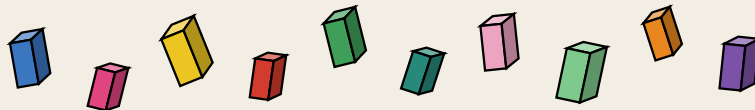
PRINT #ch;A\$ writes a string and appends an EOL; **PRINT #ch;A\$;** (trailing semicolon) writes without one; **PUT #ch,B** writes one byte. NDEV **buffers** what you write into a 128-byte transmit buffer and sends it in bunches — you will meet that buffer, and the command that flushes it early, in the next chapter.

LISTING 2. POST TO A TCP SERVICE

```
10 REM --- OPEN A TCP SOCKET, TALK, LISTEN ---
20 DIM M$(100),R$(255)
30 OPEN #1,12,0,"N1:TCP://192.168.1.5:9000/"
40 M$="HELLO FROM THE ATARI"
50 PRINT #1;M$:REM SENDS M$ + EOL
60 REM --- NOW READ THE REPLY ---
70 STATUS #1,S:BW=PEEK(746)+PEEK(747)*256
80 IF BW=0 AND PEEK(748)=1 THEN GOTO 70
90 IF BW=0 THEN GOTO 200
100 INPUT #1,R$:PRINT R$:GOTO 70
200 CLOSE #1
```

CLOSING

CLOSE #ch tears down the connection. NDEV first **flushes** anything still in the transmit buffer, then tells the FujiNet to close the protocol. Always close a channel you opened — the handler keeps only eight, one per unit, and a stranded socket ties one up until the next cold start (or a **CLR/END**, which BASIC turns into closes).



THE XIO COMMANDS

`OPEN`, `INPUT`, `PRINT`, `GET`, `PUT`, `STATUS` and `CLOSE` cover reading and writing. Everything **else** the Network device can do — make a directory, delete a file, parse JSON, set a password, flush the buffer — is a CIO **special** command, issued from BASIC with `XIO`.

HOW XIO REACHES NDEV

`XIO cmd,#ch,aux1,aux2,"Nx:..."` sends command number **cmd** to the handler. CIO routes any command of 14 or more to the handler's **special** entry, and NDEV does something clever with it:

- 1 If **cmd** is **15**, NDEV handles it itself — the put-buffer flush, below. It never reaches the wire.
- 2 Otherwise NDEV **asks the FujiNet** what that command expects: does it send data, receive data, or neither? (It does this with a one-byte inquiry.) If the FujiNet answers "I don't know that command," NDEV returns **error 146**, unimplemented.
- 3 If the command is known, NDEV issues it — using the **command number itself as the SIO command byte** — passing your `aux1/aux2` through, and the device-spec string as a 256-byte payload when the command sends data.

The consequence worth remembering: **the XIO command number equals the SIO command byte**. `XIO 42` sends SIO command `$2A` — which is `*`, make-directory. You can read the whole table below as ATASCII if you like.

XIO 15 — FLUSH THE PUT BUFFER

FLUSH TRANSMIT BUFFER

N: — XIO 15

NDEV collects your `PUT`s and `PRINT`s in a 128-byte transmit buffer and sends them only when the buffer fills, when an EOL (`$9B`) goes by, or when the channel closes. **XIO 15 sends whatever is buffered right now** — before the buffer is full and before any EOL. This is implemented **inside NDEV itself**; no other special command is.

Reach for it whenever you must get bytes out immediately: a protocol that has no line endings, a prompt you must send without a newline, an interactive exchange where the far end is waiting on your half of a line.

RETURNS `1` always — the flush cannot fail. On the open write channel only.

LISTING 3. SEND A PARTIAL LINE, THEN FLUSH

```
10 OPEN #1,12,0,"N1:TCP://10.0.0.9:23/"
20 REM SEND A PROMPT WITH NO NEWLINE...
30 PRINT #1;"LOGIN: ";
40 REM ...AND PUSH IT OUT NOW:
50 XIO 15,#1,0,0,"N:"
60 REM THE FAR END SEES "LOGIN: " IMMEDIATELY
```

THE COMPLETE XIO COMMAND SET

Every special command NDEV can reach is listed here. **cmd** is the number you give `XIO` (and the SIO command byte, in hex). Where a command sends a string, put it in the `XIO` filespec; where it takes numbers, pass them in `aux1/aux2`. Filesystem commands may be issued on any free IOCB (the device spec carries the target); connection commands (**parse, query, translation, channel mode, close-client, accept, flush**) act on an already-`OPEN` channel.

XIO	SIO	Char	Command — what it does
15	—	—	flush the transmit buffer now (handled inside NDEV)
32	\$20		rename a file; filespec is "Nx:from,to"
33	\$21	!	delete a file
35	\$23	#	lock a file (make it read-only)
36	\$24	\$	unlock a file
42	\$2A	*	make a directory

43	\$2B	+	remove a directory
44	\$2C	,	change directory (set the channel's path prefix)
48	\$30	0	get current directory (reads the prefix back)
65	\$41	A	TCP: accept a waiting client on a listening channel
68	\$44	D	UDP: set the destination "host:port" for writes
80	\$50	P	JSON: parse the document just read
81	\$51	Q	JSON: set a query path; the value becomes readable
84	\$54	T	set the translation mode (aux2 = 0/1/2/3)
90	\$5A	Z	set the interrupt/status poll rate (aux1/aux2)
99	\$63	c	TCP: close the current client, keep listening
251	\$FB	-	set a JSON parameter (aux1 selects which)
252	\$FC	-	set channel mode (aux2: 0 = protocol, 1 = JSON)
253	\$FD	-	set the username (for FTP, SMB) — filespec is the name
254	\$FE	-	set the password — filespec is the password

WORKING WITH FILES

The filesystem commands each take a device spec naming the target, on any free IOCB.

LISTING 4. DIRECTORY HOUSEKEEPING OVER TNFS

```
10 REM MAKE, RENAME, DELETE ON A TNFS HOST
20 XIO 42,#6,0,0,"N:TNFS://192.168.1.10/SAVES"
30 REM ^ MAKE DIRECTORY /SAVES
40 XIO 32,#6,0,0,"N:TNFS://192.168.1.10/A.DAT,B.DAT"
50 REM ^ RENAME A.DAT TO B.DAT (COMMA SEPARATES)
60 XIO 33,#6,0,0,"N:TNFS://192.168.1.10/B.DAT"
70 REM ^ DELETE B.DAT
80 XIO 43,#6,0,0,"N:TNFS://192.168.1.10/SAVES"
90 REM ^ REMOVE DIRECTORY /SAVES
```

SETTING CREDENTIALS

For FTP and SMB, set the username and password **before** you **OPEN** the resource.

LISTING 5. LOG IN TO AN FTP HOST

```
10 XIO 253,#1,0,0,"N1:anonymous"
20 XIO 254,#1,0,0,"N1:guest@example.com"
30 OPEN #1,4,2,"N1:FTP://ftp.example.com/readme.txt"
```

READING JSON

FujiNet can pick a single value out of a JSON document for you, so a BASIC program need never parse braces. Suppose the resource at `status.json` returns this document:

LISTING 6. THE JSON DOCUMENT RETURNED BY STATUS.JSON

```
{
  "status": {
    "code": 200,
    "message": "All systems go"
  },
  "hosts": ["fujinet.online", "irata.online"]
}
```

Open the resource, switch the channel into JSON mode, parse it, set a JSONPath, and read the value with an ordinary **INPUT**:

LISTING 7. PULL ONE FIELD OUT OF A JSON REPLY

```
10 DIM V$(255)
20 OPEN #1,4,0,"N:HTTPS://api.example.com/status.json"
30 XIO 252,#1,0,1,"N:":REM CHANNEL MODE = JSON
40 XIO 80,#1,0,0,"N:":REM PARSE THE DOCUMENT
50 XIO 81,#1,0,0,"N:/status/message":REM QUERY PATH
60 INPUT #1,V$:REM THE VALUE OF /status/message
70 PRINT V$:REM PRINTS: ALL SYSTEMS GO
80 CLOSE #1
```

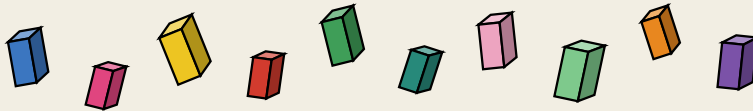
The path after the colon is a JSONPath-style selector: object members are names, array indices are numbers. Against the document above, `/status/message` selects the string `All systems go`; `/hosts/0` would select `fujinet.online`, and `/status/code` the number `200`. After `XIO 81` the value is waiting in the channel — read it as text with `INPUT`, or byte-by-byte with `GET`.

TCP SERVERS AND UDP

A listening TCP channel accepts clients with `XIO 65` and drops the current one with `XIO 99`. A UDP channel aims its writes with `XIO 68`.

LISTING 8. ANSWER A TCP CLIENT; AIM A UDP DATAGRAM

```
10 REM --- LISTEN AND ACCEPT (TCP SERVER) ---
20 OPEN #1,12,0,"N1:TCP://:9000/":REM NO HOST = LISTEN
30 XIO 65,#1,0,0,"N:":REM ACCEPT A WAITING CLIENT
40 REM ...CONVERSE ON #1... XIO 99 DROPS THE CLIENT
50 REM
60 REM --- AIM A UDP DATAGRAM ---
70 OPEN #2,12,0,"N2:UDP://:5000/"
80 XIO 68,#2,0,0,"N2:192.168.1.50:5001"
90 PRINT #2;"PING":REM GOES TO 192.168.1.50:5001
```



TALKING TO SIO DIRECTLY

Below NDEV, below CIO, is the bus itself. Every FujiNet command in the two chapters that follow is, underneath, a **SIO transaction**: you fill a Device Control Block and call the OS. This chapter is the one primitive the rest of the low road is built on.

THE DEVICE CONTROL BLOCK

The DCB is twelve bytes of page three, from `$0300`. You fill it in, call `SIOV`, and read the result back out of `DSTATS`.

Addr	Name	Holds
\$0300	DDEVIC	device id — \$71 network, \$70 Fuji, \$45 clock
\$0301	DUNIT	unit — the channel; the OS sends DDEVIC+DUNIT-1 on the wire
\$0302	DCOMND	command byte
\$0303	DSTATS	before: data direction; after: the result code
\$0304	DBUFLO/HI	buffer address (payload in or out)
\$0306	DTIMLO	timeout, in seconds — \$1F (31) is ample
\$0308	DBYTL0/HI	byte count of the payload
\$030A	DAUX1/2	the two auxiliary bytes

DSTATS, going in, tells SIO which way the data flows:

Value	Direction
\$80	Atari → FujiNet — the payload at DBUF is sent
\$40	FujiNet → Atari — the reply is read into DBUF
\$00	no payload — command and aux bytes only

Coming back out, **DSTATS** is the result. **1** is success; the OS reports timeouts and bus trouble with the codes below. **The FujiNet's own errors do not appear here** — a value of **144** means “the device signalled an error,” and you must then issue a **STATUS** command and read the extended error out of the status reply (Chapter 6).

DSTATS	Meaning
1	success
138	device timed out — no answer on the bus
139	device NAK — the FujiNet refused the frame
143	bad frame — checksum mismatch
144	device error — issue STATUS for the real cause

THE COMMAND FRAME, AND WHAT SIO DOES WITH IT

When you call `SIOV`, the OS builds a five-byte **command frame** — device id, command, `aux1`, `aux2`, checksum — and clocks it out. The FujiNet answers each stage with a single byte: **ACK** (`$41`, `'A'`) or **NAK** (`$4E`, `'N'`) for the frame, then, after the work is done, **COMPLETE** (`$43`, `'C'`) or **ERROR** (`$45`, `'E'`). Payload bytes flow in whichever direction **DSTATS** asked, guarded by their own checksum. You never see these bytes yourself — the OS folds them into the **DSTATS** result — but knowing the shape of the handshake explains every value the bus can return.

A REUSABLE SIO CALL

Rather than poke twelve addresses at every command, keep the DCB body as a template and copy it in. This is exactly what NDEV does internally; here it is in the small, in 6502 that assembles under `ca65` or MADS.

LISTING 9. SIOCALL — COPY A TEMPLATE TO THE DCB AND GO

```
DDEVIC = $0300      ; the Device Control Block
SIOV   = $E459      ; OS serial-I/O entry
DVSTAT = $02EA      ; 4-byte status buffer

; -----
```

```

; SIOCALL: A/Y point at a 12-byte DCB template.
; copies it into the real DCB at $0300, calls SIOV,
; returns the result code in A (and in DSTATS/$0303).
; -----
SIOCALL stx tmpx+1      ; (save nothing; X is free here)
        sta src+1
        sty src+2
        ldy #11         ; 12 bytes: $0300..$030B
copy    lda $FFFF,y      ; src, patched just above
src     = copy+1
        sta DDEVIC,y
        dey
        bpl copy
        jsr SIOV         ; do it
        lda $0303        ; DSTATS -> A
tmpx    ldx #0
        rts

```

The template is the twelve DCB bytes laid out in order. Here is the one that reads the network device's status — you will reuse the shape for every command that follows.

LISTING 10. A DCB TEMPLATE: NETWORK STATUS ON N1:

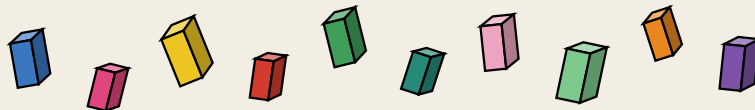
```

; DDEVIC DUNIT DCOMND DSTATS DBUF DTIM RES DBYT DAUX
STATCB .byte $71      ; DDEVIC : network device
        .byte $01      ; DUNIT : channel 1 (wire id $71)
        .byte $53      ; DCOMND : 'S' status
        .byte $40      ; DSTATS : FujiNet -> Atari
        .word DVSTAT    ; DBUF : into the status buffer
        .byte $1F      ; DTIMLO : 31 seconds
        .byte $00      ; (reserved)
        .word 4         ; DBYT : four bytes come back
        .byte $00,$00   ; DAUX1, DAUX2

; lda #<STATCB : ldy #>STATCB : jsr SIOCALL
; DVSTAT+0/+1 = bytes waiting, +2 = connected, +3 = error

```

Everything the low road does is that pattern: name a device in **DDEVIC/DUNIT**, a command in **DCOMND**, a direction and a buffer, and go. The two chapters that follow give the command byte, the parameters, and a worked example for every command the FujiNet knows.



THE NETWORK DEVICE SIO REFERENCE

The Network device answers at bus id \$71 for N1: through \$78 for N8: — set DDEVIC to \$71 and DUNIT to the channel. Its commands are the same bytes NDEV sends; the difference is only that here **you** fill the DCB. Each entry gives the command byte, its direction and parameters, and what it returns.

OPEN

N: – SIO \$4F 'O'

Instantiate the protocol and connect. Direction \$80 (send). DAUX1 is the access mode, DAUX2 the translation mode (the same values as BASIC OPEN; see Chapter 3). The payload is the device-spec string, padded to 256 bytes; NDEV sends a full page.

Field	Value
DCOMND	\$4F
DSTATS	\$80 (Atari → FujiNet)
DBYT	256 (the padded device spec)
DAUX1	access mode — 4 read, 8 write, 12 read/write, 13 POST, 5 DELETE
DAUX2	translation — 0 none, 1 CR, 2 LF, 3 CR/LF

RETURNS DSTATS = 1 on success. On 144, the connection failed — read the extended error with STATUS.

CLOSE

N: – SIO \$43 'C'

Tear the connection down. No payload. Direction \$00, DBYT 0.

RETURNS DSTATS = 1. Always close what you opened.

READ

N: – SIO \$52 'R'

Receive up to DAUX1 / DAUX2 (a 16-bit count, low in DAUX1) bytes into DBUF. Direction \$40. **Ask STATUS first** and never request more than is waiting, nor more than a single frame carries.

Field	Value
DCOMND	\$52
DSTATS	\$40 (FujiNet → Atari)
DBYT	the byte count you are reading
DAUX1/2	the same count, low byte / high byte

RETURNS the bytes in DBUF; DSTATS = 1, or an error if the read fell short.

WRITE

N: – SIO \$57 'W'

Send DAUX1 / DAUX2 bytes from DBUF. Direction \$80.

Field	Value
DCOMND	\$57
DSTATS	\$80 (Atari → FujiNet)
DBYT	the byte count you are writing
DAUX1/2	the same count, low / high

RETURNS DSTATS = 1 when the bytes were accepted.

STATUS

N: – SIO \$53 'S'

Read the four-byte channel status into DBUF (point it at DVSTAT, \$02EA).

Byte	Meaning
0–1	bytes waiting to be read (low / high)
2	connection: 1 = up, 0 = closed by the far end
3	device error — 1 = OK, 136 = end of file, else a code from Appendix A

RETURNS the four bytes; poll this between reads to pace a socket.

LISTING 11. HTTP GET, THE LOW ROAD

```
; open N1:HTTP://.../ for read, drain it to the screen.
; assumes SIOCALL from Chapter 5, and COUT at $F6A4 (E: put)

GET      lda #<OPENCB : ldy #>OPENCB : jsr SIOCALL
        bmi  GERR      ; DSTATS bit 7 set = trouble
LOOP     lda #<STATCB : ldy #>STATCB : jsr SIOCALL
        lda  DVSTAT+3   ; device error
        cmp  #136      ; end of file?
        beq  GDONE
        lda  DVSTAT     ; bytes waiting (low)
        ora  DVSTAT+1   ; (high)
        beq  LOOP      ; nothing yet -- poll again
        ; set READ length = min(bytes waiting, 128) -- omitted
        lda  #<READCB : ldy #>READCB : jsr SIOCALL
        ; ... emit DBUF for the byte count via COUT ...
        jmp  LOOP
GDONE    lda #<CLOSCB : ldy #>CLOSCB : jsr SIOCALL
GERR     rts

OPENCB   .byte $71,$01,$4F,$80 ; N1: OPEN, send
        .word SPEC           ; device spec buffer
        .byte $1F,$00
        .word 256            ; a full page
        .byte $04,$02        ; mode 4 (read), trans 2 (LF)
SPEC     .byte "N1:HTTP://fujinet.online/",$9B
        .res 256             ; padded out to a page
```

FILESYSTEM COMMANDS

Each names its target in the device-spec payload (direction **\$80**), just as the XIO forms do. The command bytes are the ATASCII characters in the table.

Cmd	Char	Operation	Payload is
\$21	!	delete a file	the device spec
\$20		rename a file	spec "Nx:from,to"
\$23	#	lock (read-only)	the device spec
\$24	\$	unlock	the device spec
\$2A	*	make directory	the device spec
\$2B	+	remove directory	the device spec
\$2C	,	change directory	the new prefix
\$30	0	get current dir	read the prefix into DBUF (direction \$40)

JSON, CREDENTIALS, AND CHANNEL MODE

SET CHANNEL MODE

N: – SIO \$FC

DAUX2 = 0 for the ordinary protocol channel, 1 for JSON. No payload.

PARSE JSON

N: – SIO \$50 'P'

Parse the document just read on the channel. No payload. Follow with a query.

QUERY JSON

N: – SIO \$51 'Q'

Send a JSONPath string (direction **\$80**); the selected value becomes available to **READ**. DAUX2 sets the value's translation (0/1/2).

USERNAME / PASSWORD

N: – SIO \$FD / \$FE

Send the credential string (direction **\$80**) **before** **OPEN**. \$FD sets the username, \$FE the password — for FTP and SMB.

TRANSLATION

N: — SIO \$54 'T'

Set the channel's end-of-line translation from **DAUX2** (0/1/2/3). No payload. Applies to subsequent opens on the channel.

SET INTERRUPT RATE

N: — SIO \$5A 'Z'

Set how often the FujiNet raises the PROCEED interrupt to offer new status, as a 16-bit millisecond count in **DAUX1** (low) / **DAUX2** (high). No payload.

TCP AND UDP

ACCEPT CLIENT

N: — SIO \$41 'A'

On a listening TCP channel, accept a waiting client. No payload.

CLOSE CLIENT

N: — SIO \$63 'c'

Drop the current TCP client but keep listening. No payload.

SET UDP DESTINATION

N: — SIO \$44 'D'

Send "host:port" (direction **\$80**); subsequent writes go there.

GET UDP REMOTE

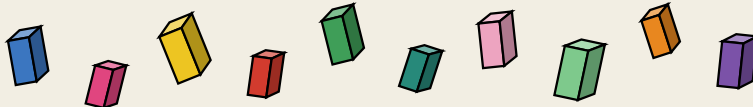
N: — SIO \$72 'r'

Read the address of the last datagram's sender into **DBUF** (direction **\$40**). **This command is reachable only on the low road — see Chapter 9.**

INQUIRE DIRECTION

N: — SIO \$FF

Ask what data direction a command uses. **DAUX1** is the command byte to ask about; the reply is one byte: **\$00** none, **\$40** read, **\$80** write, **\$FF** unknown. This is the very call NDEV makes before every special command; you can use it to write a handler of your own.



THE FUJI CONTROL DEVICE

SIO REFERENCE

The device at bus id `$70` is the one CONFIG talks to: the WiFi radio, the host and disk-image mounts, the directory browser, the app-key store, the clock, and a shelf of utilities. **NDEV does not expose any of it** — this whole chapter is the low road only. Set `DDEVIC` to `$70`, `DUNIT` to `1`.

A note on parameters: unlike the Network device, the Fuji device takes most of its small arguments in `DAUX1/DAUX2` and reserves the payload for bulk data (names, slot arrays, disk images). Each entry says which.

WIFI AND THE ADAPTER

SCAN NETWORKS

Fuji – SIO `$FD`

Kick off a scan. Read one byte back: the count of access points found. Direction `$40`, `DBYT` `1`.

LISTING 12. SCAN, THEN LIST THE ACCESS POINTS

```
; DDEVIC=$70 throughout. A count comes back in NBUF.
SCAN    lda #<SCANCB : ldy #>SCANCB : jsr SIOCALL
        lda NBUF          ; A = number of APs
        sta COUNT
        ; --- for each index 0..COUNT-1, GET SCAN RESULT ---
        ldx #0
SR       stx RESCB+10      ; DAUX1 = index
        lda #<RESCB : ldy #>RESCB : jsr SIOCALL
        ; ENTRY now holds 32-byte SSID + 1 signed RSSI byte
        ; ... print it ...
        inx : cpx COUNT : bne SR
        rts

SCANCB   .byte $70,$01,$FD,$40 : .word NBUF
        .byte $1F,$00 : .word 1 : .byte $00,$00
RESCB    .byte $70,$01,$FC,$40 : .word ENTRY
        .byte $1F,$00 : .word 33 : .byte $00,$00
NBUF     .res 1
ENTRY    .res 33
```

GET SCAN RESULT

Fuji – SIO `$FC`

Read one scanned access point. `DAUX1` is the index (0-based). Reads a 33-byte record: a 32-byte SSID followed by one signed RSSI byte. Direction `$40`.

SET SSID

Fuji – SIO `$FB`

Join a network. Payload (direction `$80`) is a 32-byte SSID then a 64-byte password; `DAUX1` = 1 stores it to config so the FujiNet rejoins at power-on.

GET SSID

Fuji – SIO `$FE`

Read the stored SSID and password (same 96-byte layout). Direction `$40`.

GET WIFI STATUS

Fuji – SIO `$FA`

Read one byte: `3` = connected, `6` = not connected. Direction `$40`.

GET ADAPTER CONFIG

Fuji – SIO `$E8`

Read the live network configuration into `DBUF`. Direction `$40`.

Off	Bytes	Field
-----	-------	-------

0	32	SSID
32	64	hostname
96	4	local IP
100	4	gateway
104	4	netmask
108	4	DNS server
112	6	MAC address
118	6	BSSID
124	15	firmware version string

HOSTS AND DISK SLOTS

Host slots (8) name where disks live — a TNFS server, an SMB share, the SD card. **Disk slots** (8) are the drive bays **D1:** through **D8:**. Mounting is two steps: mount a host, then mount an image from it into a disk slot.

Cmd	Operation	Parameters
\$F4	read host slots	read 8 × 32-byte names (256 bytes), direction \$40
\$F3	write host slots	send 256 bytes, direction \$80
\$F2	read device slots	read the 8 × 38-byte slot array, direction \$40
\$F1	write device slots	send the slot array, direction \$80
\$F9	mount host	DAUX1 = host slot
\$E6	unmount host	DAUX1 = host slot
\$F8	mount image	DAUX1 = disk slot, DAUX2 = mode (1 RO, 2 RW)
\$E9	unmount image	DAUX1 = disk slot
\$D7	mount all	no parameters — mount every configured slot
\$E2	set device filename	DAUX1 = slot, DAUX2 = host<<4 mode; payload = name
\$A0-\$A9	get device filename	\$A0 + slot; read the path back, direction \$40

A **device-slot record** is 38 bytes: host slot (1), access mode (1: 1 = read, 2 = read/write), then a 36-byte filename.

LISTING 13. MOUNT HOST SLOT 0, IMAGE INTO DISK SLOT 1 (R/W)

```

MOUNT    lda #<MHOST : ldy #>MHOST : jsr SIOCALL    ; mount host
          lda #<MIMG  : ldy #>MIMG  : jsr SIOCALL    ; mount image
          rts
; mount host: DAUX1 = host slot 0
MHOST    .byte $70,$01,$F9,$00 : .word 0
          .byte $1F,$00 : .word 0 : .byte $00,$00
; mount image: DAUX1 = disk slot 1, DAUX2 = 2 (read/write)
MIMG     .byte $70,$01,$F8,$00 : .word 0
          .byte $1F,$00 : .word 0 : .byte $01,$02

```

NEW DISK

Fuji – SIO \$E7

Manufacture a blank disk image on a writable host. Direction **\$80** the payload is a short header then the filename:

Off	Bytes	Field
0	2	number of sectors (low / high)
2	2	sector size (128 / 256 / 512)
4	1	host slot
5	1	device (disk) slot
6	256	filename

BROWSING A HOST

OPEN DIRECTORY

Fuji – SIO \$F7

DAUX1 = host slot; payload (direction **\$80**) is the path, NUL, then an optional wildcard filter.

READ DIRECTORY ENTRY

Fuji — SIO \$F6

DAUX1 = maximum length to return; DAUX2 = flags (\$80 appends packed date/size/flags after the name). Reads one entry into DBUF. A first byte of \$7F is the end-of-directory marker.

CLOSE DIRECTORY

Fuji — SIO \$F5

No parameters.

GET / SET POSITION

Fuji — SIO \$E5 / \$E4

Read the current directory position (2 bytes), or set it from DAUX1 / DAUX2, for paging.

APP KEYS — A PLACE TO KEEP STATE

The FujiNet will hold a small block (up to 64 bytes) on its SD card for your program, indexed by a **creator id**, **app id** and **key id**. Ideal for high scores, saved options, a resume point.

OPEN APP KEY

Fuji — SIO \$DC

Payload (direction \$80): creator id (2 bytes, low first), app id (1), key id (1), mode (1: 0 = read, 1 = write). Opens the key for the read or write that follows.

WRITE APP KEY

Fuji — SIO \$DE

After opening for write, send the data. DAUX1 / DAUX2 carry the length (16-bit, low first). Direction \$80 up to 64 bytes are stored.

READ APP KEY

Fuji — SIO \$DD

After opening for read, read the block back (a 2-byte length then the data). Direction \$40.

CLOSE APP KEY

Fuji — SIO \$DB

No parameters — closes the current key.

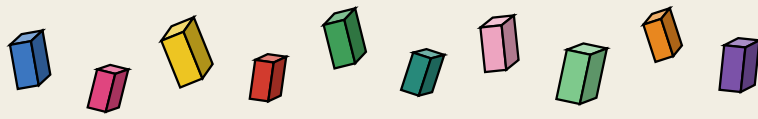
THE CLOCK, UTILITIES, AND HOUSEKEEPING

The Fuji device also tells time (Chapter 8) and carries a shelf of utilities. Small ones:

Cmd	Operation	Notes
\$FF	reset the FujiNet	no parameters
\$D9	enable/disable CONFIG boot	DAUX1 = 0/1
\$D6	set boot mode	DAUX1 = mode
\$D5 / \$D4	enable / disable a device	DAUX1 = device id
\$D1	device-enabled status	read one byte, direction \$40
\$D8	copy a file between hosts	payload = source, dest slots + spec
\$D3	random number	read 4 bytes, direction \$40
\$BB	generate a GUID	read the 36-char string, direction \$40
\$EB	set SIO baud rate	DAUX1 = index (0 = 19200 ... 6 = 921600)
\$E3	set high-speed SIO index	DAUX1 = index, DAUX2 = 1 to save

And three multi-step utility families — each is **input**, **compute**, **length**, **output**, sending data in and reading the result back out in pieces:

Family	Commands	Notes
Hash	\$C8 \$C7 \$C6 \$C5 \$C2	input, compute, length, output, clear; algorithm in the byte after \$C7 (0 MD5, 1 SHA-1, 2 SHA-256, 3 SHA-512)
Base64 encode	\$D0 \$CF \$CE \$CD	input, compute, length, output
Base64 decode	\$CC \$CB \$CA \$C9	input, compute, length, output
QR code	\$BC \$BD \$BE \$BF	input, encode, length, output — the bytes of a QR bitmap



TELLING TIME

The Atari has never known what time it is. The FujiNet does — it keeps the clock from the Internet — and there are two ways to ask.

FROM THE FUJI DEVICE

GET TIME

Fuji – SIO \$D2

Point **DDEVIC** at **\$70**, send **\$D2**, and read seven bytes: a binary date and time in the FujiNet's configured zone.

Off	Bytes	Field
0	1	century (add to year: \$13 = 19 → 1900s, \$14 = 20 → 2000s)
1	1	year (0–99)
2	1	month (1–12)
3	1	day
4	1	hour (24-hour)
5	1	minute
6	1	second

LISTING 14. READ THE WALL CLOCK

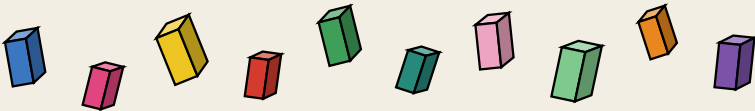
```
GETTIME lda #<TIMECB : ldy #>TIMECB : jsr SIOCALL
        ; NOW+0..6 = century, year, month, day, hour, min, sec
        rts
TIMECB .byte $70,$01,$D2,$40 : .word NOW
        .byte $1F,$00 : .word 7 : .byte $00,$00
NOW .res 7
```

FROM THE CLOCK (APETIME) DEVICE

FujiNet also emulates the venerable **APETime** clock at bus id **\$45**, the interface a good deal of existing Atari software already understands. It offers the time pre-formatted several ways, chosen by the command byte:

Cmd	Format
\$93	APETime binary (6 bytes: DD MMY HH MM SS)
\$41 'A'	Atari-native binary
\$49 'I'	ISO-8601 local time, as a string
\$5A 'Z'	ISO-8601 UTC, as a string
\$50 'P'	ProDOS date/time
\$99	set the time zone (payload = TZ string)

The clock device is the right target when adapting software that already speaks APETime; the Fuji device's **\$D2** is the simplest when writing fresh.



WHAT NDEV CANNOT REACH

The high road is broad but not complete. Some of the FujiNet is out of NDEV's sight, and knowing exactly where the road ends saves an afternoon of wondering why an **XIO** returns error 146. Each gap below is also a standing invitation: NDEV is open source (Chapter 2, Appendix C), and closing these is the obvious next work.

THE WHOLE FUJI CONTROL DEVICE

This is the big one. **NDEV speaks only to the Network device** (**\$71–\$78**). Everything in Chapter 7 — mounting hosts and disks, scanning WiFi, reading the clock, the app-key store, the directory browser, hashing, Base64, QR codes — lives on the Fuji device (**\$70**), which NDEV never addresses. From BASIC there is no **XIO** that will mount a disk. To reach the Fuji device you must take the low road: poke the DCB and call **SIOV** yourself (which BASIC can do through **USR** and a short machine-code stub, or a language cartridge can do directly).

A worthy improvement: a companion handler, or an extension to NDEV, that surfaces the common Fuji commands as **XIO**s on a pseudo-device — mount, unmount, scan, get-time — so a BASIC program could manage its own disks.

A HANDFUL OF NETWORK SUB-COMMANDS

NDEV reaches a network special command only if the FujiNet's **inquiry** (Chapter 6, **\$FF**) reports a known data direction for it. A few commands are not in that table, so NDEV answers **error 146** when you try:

SIO	Char	Command	Why it is out of reach
\$72	r	UDP get remote address	not in the inquiry table — reachable only via direct SIO
\$4D	M	HTTP set channel mode (headers)	not in the inquiry table; set the verb via OPEN aux1 instead
\$FA	—	set channel (advanced)	not surfaced through NDEV

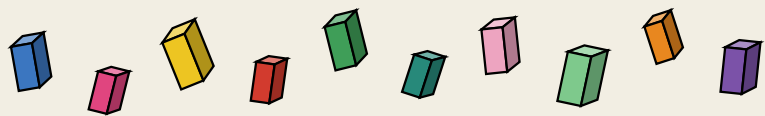
None of these is fatal — the UDP sender address you can read on the low road, and HTTP verbs you select through the **OPEN** access mode — but they are honest gaps. Adding their rows to the firmware's inquiry table would let NDEV pass them straight through.

THE 128-BYTE RECEIVE CAP

NDEV's receive path caps a single read at **127 bytes** (its transmit and receive buffers are 128 bytes each). A program that reads through **GET/INPUT** never notices — the handler loops — but it means NDEV cannot hand back a 512-byte block in one call the way a direct **READ** can. For bulk transfer, the low road's larger single reads are faster. **An improvement:** a larger, page-aligned buffer, or a block-read special command.

STATUS IS FOUR BYTES, NO MORE

The handler's **STATUS** returns exactly the four **DVSTAT** bytes — bytes-waiting, connection, error. Richer per-protocol status (an HTTP response code, an FTP reply line) is available on the wire but not surfaced by NDEV. Reading it is a low-road affair today.



OTHER LANGUAGES THROUGH CIO

Nothing about the N: handler is particular to Atari BASIC. NDEV installs itself into the Atari's handler table, and **any language that can reach CIO can reach the FujiNet**. If it can **OPEN** a device and **GET / PUT** bytes, it can talk to **N:**.

THE ASSEMBLER / EDITOR CARTRIDGE

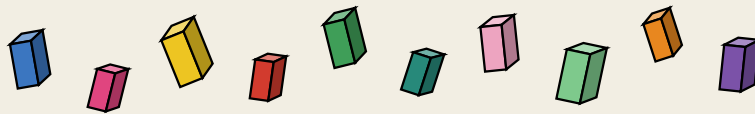
The Atari Assembler/Editor cartridge (and the Macro Assembler, and MAC/65) assemble programs that call **CIOV** at **\$E456** directly. Set up an IOCB — handler-id from the device name, command, buffer, aux bytes — load **X** with the IOCB number times sixteen, and **JSR CIOV**. The special commands of Chapter 4 are just command bytes 14 and up in **ICCOM**. A program written this way needs no BASIC at all, and runs at assembly speed while still riding NDEV's buffering and interrupt handling.

ATARI LOGO

Atari Logo has no file words of its own, but it has three that let you reach anything: the low-level primitives **.DEPOSIT**, **.EXAMINE**, and **.CALL**. With them you poke a tiny machine-code stub into page six (**\$0600**, free on every Atari), set up an IOCB, and **.CALL CIOV** through the stub. The next chapter shows exactly this.

AND THE REST

The same door is open to Action!, to Forth, to C compiled with cc65, to Pascal — to any language on the Atari that can call the OS. Most reach **N:** through CIO exactly as BASIC does; some (Action!, in Chapter 12) skip NDEV entirely and drive SIO. The point stands: **the N: handler is a public utility, and every language is welcome to it.**



FUJINET FROM ATARI LOGO

Atari Logo is turtles and lists, not file handles — but its three low-level primitives open a path to the N: handler. The plan: build a three-byte stub in page six that selects an IOCB and jumps to **CIOV**, then drive an IOCB from Logo with **.DEPOSIT**, calling the stub with **.CALL** for each CIO operation.

THE STUB

CIO expects the IOCB number, times sixteen, in the **X** register; **.CALL** enters an address with a plain **JSR** and does not set registers. So we lay down a stub that sets **X** itself and jumps to **CIOV** (**\$E456**, decimal 58454):

```
$0600 · LDX #$10 — 162 16 — use IOCB #1
$0602 · JMP CIOV — 76 86 228
```

LISTING 15. REACH N: FROM ATARI LOGO

```
TO NSETUP
; --- lay the stub at 1536 ($0600) ---
.DEPOSIT 1536 162 ; LDX #$10 (IOCB 1)
.DEPOSIT 1537 16
.DEPOSIT 1538 76 ; JMP $E456 (CIOV)
.DEPOSIT 1539 86
.DEPOSIT 1540 228
END

TO NPUTSPEC :SPEC :ADDR ; the device spec, char by char
IF EMPTY :SPEC [.DEPOSIT :ADDR 155 STOP] ; 155 = EOL
.DEPOSIT :ADDR ASCII FIRST :SPEC
NPUTSPEC BUTFIRST :SPEC :ADDR + 1
END

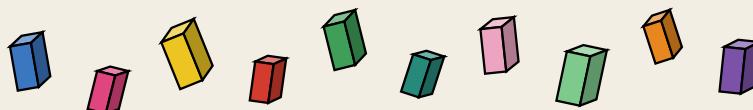
TO NOPEN :SPEC
NPUTSPEC :SPEC 1280 ; device spec into $0500
; IOCB 1 is at $0350..$035F (848..)
.DEPOSIT 850 3 ; ICCOM = 3 (OPEN)
.DEPOSIT 852 0 .DEPOSIT 853 5 ; ICBAL/H = $0500
.DEPOSIT 858 4 ; ICAX1 = 4 (read)
.DEPOSIT 859 2 ; ICAX2 = 2 (LF)
.CALL 1536
END

TO NGET ; read one byte (GET CHARACTERS)
.DEPOSIT 850 7 ; ICCOM = 7
.DEPOSIT 852 0 .DEPOSIT 853 5 ; ICBAL/H = $0500
.DEPOSIT 856 1 .DEPOSIT 857 0 ; ICBLL/H = 1 byte
.CALL 1536
OUTPUT .EXAMINE 1280 ; the byte at $0500
END

TO NCLOSE
.DEPOSIT 850 12 ; ICCOM = 12 (CLOSE)
.CALL 1536
END
```

IOCB #1's control block begins at **\$0350** (decimal 848): command at +2 (850), buffer address at +4/+5 (852/853), buffer length at +8/+9 (856/857), the aux bytes at +10/+11 (858/859). **.DEPOSIT address value** writes a byte; **.EXAMINE address** reads one back. Change the stub's **LDX** operand (**16** for IOCB 1, **32** for 2, **48** for 3 ...) to drive other channels.

This is deliberately close to the metal — but it works, and it is the honest way to give a turtle a network. Anything BASIC does with **N:**, Logo can do this way; only the plumbing is longer.



FUJINET IN ACTION!

The OSS **Action!** language compiles to fast 6502 and lets you reach memory and the OS directly — so the natural way to use the FujiNet from Action! is the **low road**: fill the DCB, call **SI0V**, skip NDEV entirely. This chapter walks the **NIO.ACT** library, a small Action! module that does exactly that. (The full source is at the end of this chapter; the companion **Action! Reference Manual** is in the repository's **learn/** folder.)

THE SHAPE OF IT

Action! lets you name the DCB fields as **BYTE** and **CARD** (16-bit) variables at their page-three addresses, and declare **SI0V** as a **PROC** at **\$E459**. After that a command is just: set the fields, call **siov()**, read **DSTATS**.

LISTING 16. NIO.ACT — THE DCB, AND THE SIO ENTRY

```
; the Device Control Block, named field by field
BYTE DDEVIC = $0300 ; Device #
BYTE DUNIT  = $0301 ; Unit #
BYTE DCOMND = $0302 ; Command
BYTE DSTATS = $0303 ; direction / result
CARD DBUF   = $0304 ; buffer
BYTE DTIMLO = $0306 ; timeout secs
CARD DBYT   = $0308 ; payload length
CARD DAUX   = $030A ; aux1 / aux2
BYTE DAUX1  = $030A
BYTE DAUX2  = $030B

PROC siov=$E459() ; the OS SIO vector
```

OPEN, READ, WRITE

Each library routine is the DCB pattern with the command byte and direction filled in. Note the direction constants — **\$40** to receive, **\$80** to send — exactly as Chapter 5 described.

LISTING 17. NIO.ACT — OPEN AND READ

```
BYTE FUNC nopen(BYTE ARRAY ds, BYTE t)
  DDEVIC = $71
  DUNIT  = ngetunit(ds)
  DCOMND = 'O ; OPEN
  DSTATS = $80 ; Atari -> FujiNet
  DBUF   = ds ; the device spec
  DTIMLO = $1F
  DBYT   = 256
  DAUX1  = 12 ; read/write
  DAUX2  = t ; translation
  siov()
  RETURN (geterror(ds))

BYTE FUNC nread(BYTE ARRAY ds, BYTE ARRAY buf, CARD len)
  DDEVIC = $71
  DUNIT  = ngetunit(ds)
  DCOMND = 'R ; READ
  DSTATS = $40 ; FujiNet -> Atari
  DBUF   = buf
  DTIMLO = $1F
  DBYT   = len
  DAUX   = len
  siov()
  RETURN (geterror(ds))
```


THE ERROR DANCE, AND THE INTERRUPT

NIO.ACT handles errors just as NDEV does: when **DSTATS** comes back **144**, it issues a **STATUS ('S')** and reads the extended error from **DVSTAT+3**. It also carries an optional **PROCEED** interrupt handler — a five-byte routine poked into memory — that sets a flag when the FujiNet signals, so a program can await data without a busy loop, the same mechanism NDEV uses.

LISTING 18. NIO.ACT — EXTENDED ERROR, AND THE PROCEED TRIP

```
BYTE FUNC geterror(BYTE ARRAY ds)
  BYTE errno
  IF DSTATS=144 THEN      ; device-signalled error
    nstatus(ds)           ; do a STATUS
    errno=EXTERR          ; DVSTAT+3 (the real cause)
  ELSE
    errno=DSTATS
  FI
RETURN (errno)

PROC ninterrupt_handler=*( ) ; the 5-byte PROCEED handler
[$A9$01$8D trip $68$40]     ; LDA #1 : STA trip : PLA : RTI
```

WHY ACTION! TAKES THE LOW ROAD

Action! could call CIO and use NDEV — but it does not need to. It reaches the OS as easily as BASIC reaches a variable, so driving SIO directly is no harder and a good deal faster: no handler to load, no DUP to trip over, and the full Network device at hand. **NIO.ACT** is the template — copy it, point **DDEVIC** at **\$70**, and the Fuji device of Chapter 7 is yours from Action! too.

The complete, unabridged **NIO.ACT**:

LISTING 19. NIO.ACT — THE COMPLETE LIBRARY

```
;
; #FujiNet Network I/O Library
; For ACTION!
;
; Author: Thomas Cherryhomes
; <thom.cherryhomes@gmail.com>
;
; These routines call the #Fujinet
; directly from SIO, and thus do
; not need the N: handler (NDEV)
;

MODULE

;
; PROCEED interrupt vars
;
CARD VPRCEDSAVE      ; Save vector for vprced
CARD VPRCED = $0202 ; Proceed vector
BYTE PACTL = $D302 ; PIA Control for Proceed
BYTE trip           ; Trip FLAG.

;
; DVSTAT (Status)
;
BYTE DVSTAT = $02EA ; PTR TO DVSTAT
BYTE EXTERR = $02ED ; DVSTAT+3

;
; DEVICE CONTROL BLOCK (DCB)
;
BYTE DDEVIC = $0300 ; Device #
BYTE DUNIT = $0301 ; Unit #
BYTE DCOMND = $0302 ; Command
BYTE DSTATS = $0303 ; <-> and error
CARD DBUF = $0304 ; buffer
BYTE DTIMLO = $0306 ; timeout secs
BYTE DUNUSE = $0307 ; reserved
CARD DBYT = $0308 ; pyld byte len
```

```

CARD DAUX   = $030A ; daux1/daux2
BYTE DAUX1  = $030A ; daux1
BYTE DAUX2  = $030B ; daux2

;
; Interrupt handler
;
; A9 01    LDA #$01
; 8D XX XX STA trip
; 68       PLA
; 40       RTI
;
PROC ninterrupt_handler=*( )
[$A9$01$8D trip $68$40]

;
; Enable Interrupt handler.
;
PROC nenableproc()

    trip=0
    VPRCEDSAVE=VPRCED
    VPRCED=ninterrupt_handler
    PACTL = PACTL % 1

RETURN

;
; Disable Interrupt handler.
;
PROC ndisableproc()

    trip=0
    VPRCED=VPRCEDSAVE

RETURN

;
; PROC to call SIO Vector (SIOV)
;
PROC siov=$E459()

;
; Get the unit number from devicespec
;
BYTE FUNC ngetunit(BYTE ARRAY ds)
    BYTE unit=1

    IF ds(2)=': THEN
        unit=1
    ELSEIF ds(3)=': THEN
        unit=ds(2)-$30
    ELSE
        unit=1
    FI

RETURN (unit)

;
; Get status of last NIO operation,
; Return in DVSTAT
;
; @param devicespec N: devicespec
;
PROC nstatus(BYTE ARRAY ds)

    DDEVIC = $71
    DUNIT  = ngetunit(ds)
    DCOMND = 'S      ; STATUS
    DSTATS = $40      ; Payload to Atari
    DBUF   = $02EA    ; status buffer
    DTIML0 = $1F      ; 32 second timeout
    DBYT   = 4         ; 4 byte payload
    DAUX1  = 0         ; R/W
    DAUX2  = 0         ; translation
    siov()

```

```

RETURN

;
; Return error of last NIO operation.
; If SIO error = 144, then a status
; is done, and the extended err is
; returned.
;
; @param devicespec N: devicespec
; @return error 1=successful
;
BYTE FUNC geterror(BYTE ARRAY ds)
    BYTE errno

    IF DSTATS=144 THEN
        nstatus(ds)
        errno=EXTERR
    ELSE
        errno=DSTATS
    FI

RETURN (errno)

;
; Open the N: device pointed to by
; devicespec.
;
; @param devicespec N: devicespec
; @param trans - translation mode
; 0=NONE, 1=CR, 2=LF, 3=CR/LF
; @return error, 1=successful.
;
BYTE FUNC nopen(BYTE ARRAY ds, BYTE t)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = 'O' ; OPEN
    DSTATS = $80 ; Write to fujinet
    DBUF = ds ; send devicespec
    DTIMLO = $1F ; 32 second timeout
    DBYT = 256 ; 256 byte payload
    DAUX1 = 12 ; R/W
    DAUX2 = t ; translation
    siov()

; IF DSTATS=1 THEN
;     nenableproc()
; FI

RETURN (geterror(ds))

;
; Close the N: device pointed to by
; devicespec.
;
; @param devicespec N: devicespec
; @return error, 1=successful.
;
BYTE FUNC nclose(BYTE ARRAY ds)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = 'C' ; CLOSE
    DSTATS = $00 ; No Payload
    DBUF = 0 ; send devicespec
    DTIMLO = $1F ; 32 second timeout
    DBYT = 0 ; No payload
    DAUX1 = 0
    DAUX2 = 0
    siov()

; ndisableproc()

RETURN (geterror(ds))

;
; Read len bytes from N: device

```

```

; pointed to by devicespec.
;
; @param devicespec N: devicespec
; @param buf The dest buffer
; @param len # of bytes to read
; @return error 1=successful
;
BYTE FUNC nread(BYTE ARRAY ds, BYTE ARRAY buf, CARD len)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = 'R    ; READ
    DSTATS = $40    ; Atari<-Payload
    DBUF = buf    ; send devicespec
    DTIMLO = $1F    ; 32 second timeout
    DBYT = len    ; No payload
    DAUX = len
    siov()

RETURN (geterror(ds))

;
; Write len bytes to N: device
; pointed to by devicespec.
;
; @param devicespec N: devicespec
; @param buf The src buffer
; @param len # of bytes to read
; @return error 1=successful
;
BYTE FUNC nwrite(BYTE ARRAY ds, BYTE ARRAY buf, CARD len)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = 'W    ; WRITE
    DSTATS = $80    ; Payload->FujiNet
    DBUF = buf    ; send devicespec
    DTIMLO = $1F    ; 32 second timeout
    DBYT = len    ; No payload
    DAUX = len
    siov()

RETURN (geterror(ds))

; Set Channel mode for N: device
; pointed to by device spec.
;
; @param devicespec N: devicespec
; @param mode New channel mode
;
;         0 = CHANNEL,
;         1 = JSON
;
BYTE FUNC nchannelmode(BYTE ARRAY ds, BYTE mode)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = $FC
    DSTATS = $00
    DBUF = 0
    DTIMLO = $1F
    DBYT = 0
    DAUX1 = 12
    DAUX2 = mode

    siov()

RETURN (geterror(ds))

; Parse JSON document at N: ds
;
; @param devicespec N: devicespec
;
BYTE FUNC nparsejson(BYTE ARRAY ds)

    DDEVIC = $71
    DUNIT = ngetunit(ds)
    DCOMND = 'P

```

```

DSTATS = $00
DBUF    = 0
DTIMLO  = $1F
DBYT    = 0
DAUX    = 0
siov()

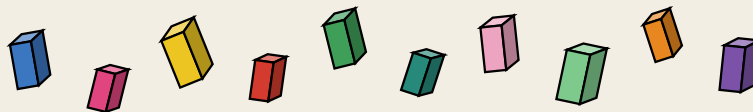
RETURN (geterror(ds))

; Query JSON path using JSON-Query
; string.
;
; @param ds N: devicespec
; @param qs JSON Query String
;
BYTE FUNC nqueryjson(BYTE ARRAY ds, BYTE ARRAY qs)

DDEVIC = $71
DUNIT  = ngetunit(ds)
DCOMND = 'Q'
DSTATS = $80
DBUF    = qs
DTIMLO  = $1F
DBYT    = 256
DAUX1   = 12
DAUX2   = 0
siov()

RETURN (geterror(ds))

```



APPENDIX A

ERROR CODES

WHAT DSTATS RETURNS FROM A SIO CALL

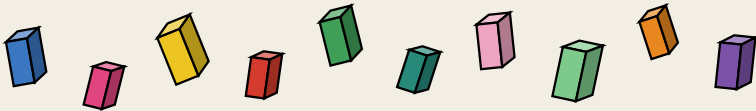
The OS leaves one of these in **DSTATS** (**\$0303**) after **SIOV**. A value with the high bit set (≥ 128) is trouble.

Code	Meaning
1	success
138	device timeout — nothing answered on the bus
139	device NAK — the frame was refused
143	bad frame — checksum mismatch
144	device error — the FujiNet signalled; read STATUS for the cause

THE FUJINET'S OWN ERROR CODES

When **DSTATS** is **144**, issue a Network **STATUS** and read byte 3 of the reply (or, through NDEV, **PEEK(749)**). These are the FujiNet device-status codes:

Code	Name	Meaning
1	SUCCESS	no error
136	END OF FILE	the resource is fully read
144	GENERAL	a fatal device error
146	NOT IMPLEMENTED	command unknown to NDEV or the device
151	FILE EXISTS	on a make-directory or create
162	NO SPACE	the device is full
165	INVALID DEVICESPEC	the N: spec could not be parsed
167	ACCESS DENIED	permission refused
170	FILE NOT FOUND	no such file, or a network 404
200	CONNECTION REFUSED	the far end refused, or is unreachable
201	NETWORK UNREACHABLE	no route to the host
202	SOCKET TIMEOUT	the connection timed out
203	NETWORK DOWN	the WiFi link is down
204	CONNECTION RESET	the far end reset the connection
207	NOT CONNECTED	operated on a closed channel
208	SERVER NOT RUNNING	a listening server returned nothing
212	BAD USER / PASSWORD	credentials rejected
213	CANNOT PARSE JSON	the document was not valid JSON
255	NO BUFFERS	the FujiNet could not allocate memory



APPENDIX B

QUICK REFERENCE

DEVICES ON THE BUS

DDEVIC	DUNIT	Wire id	Device
\$71	1–8	\$71–\$78	Network — N1: through N8:
\$70	1	\$70	Fuji control device
\$45	1	\$45	Clock (APETime)

BASIC / XIO SUMMARY (THE HIGH ROAD)

From BASIC	Does
OPEN #ch,mode,trans,"Nx:..."	connect (mode 4 read, 8 write, 12 r/w; trans 0/1/2/3)
INPUT #ch,A\$ / GET #ch,B	read a record / one byte
PRINT #ch;A\$ / PUT #ch,B	write a record / one byte
STATUS #ch,S	fill DVSTAT — PEEK(746/747) bytes, 748 conn, 749 error
CLOSE #ch	flush and disconnect
XIO 15	flush the transmit buffer now
XIO 32/33/35/36	rename / delete / lock / unlock
XIO 42/43/44/48	mkdir / rmdir / chdir / getcwd
XIO 65/99	TCP accept client / close client
XIO 68	UDP set destination
XIO 80/81	JSON parse / query
XIO 84/90	translation / interrupt rate
XIO 252	channel mode (aux2: 0 protocol, 1 JSON)
XIO 253/254	username / password

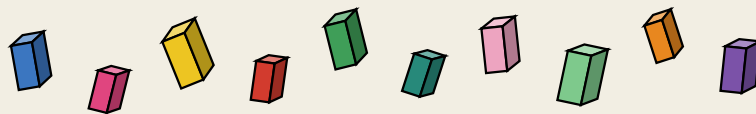
NETWORK DEVICE — SIO COMMAND BYTES (THE LOW ROAD)

Cmd	Char	Operation
\$4F	0	open connection (aux1 mode, aux2 trans, spec)
\$43	C	close connection
\$52	R	read waiting bytes (count in aux1/aux2)
\$57	W	write bytes (count in aux1/aux2)
\$53	S	channel status — 4 bytes
\$50	P	JSON parse
\$51	Q	JSON query (then read the value)
\$FC	–	channel mode (aux2: 0 protocol, 1 JSON)
\$54	T	set translation (aux2)
\$5A	Z	set interrupt rate (aux1/aux2)
\$FD / \$FE	–	set username / password
\$2C / \$30	, 0	change / get directory
\$20		rename (spec "from,to")
\$21 \$23 \$24	! # \$	delete / lock / unlock
\$2A / \$2B	* +	make / remove directory
\$41 / \$63	A c	TCP accept / close client
\$44 / \$72	D r	UDP set destination / get remote
\$FF	–	inquire a command's data direction

FUJI CONTROL DEVICE — SIO COMMAND BYTES

Cmd	Operation
\$FF	reset FujiNet

\$FD / \$FC	scan networks / get scan result n
\$FB / \$FE	set / get SSID
\$FA	get WiFi status
\$E8 / \$C4	get adapter config / extended
\$F9 / \$E6	mount / unmount host slot
\$F8 / \$E9	mount / unmount disk image
\$D7	mount all
\$F4 / \$F3	read / write host slots
\$F2 / \$F1	read / write device slots
\$E2 / \$A0-\$A9	set / get device filename
\$E7	new (blank) disk
\$F7 \$F6 \$F5	open / read / close directory
\$E5 / \$E4	get / set directory position
\$DC \$DD \$DE \$DB	app key: open / read / write / close
\$D9 / \$D6	CONFIG boot / boot mode
\$D5 / \$D4 / \$D1	enable / disable / query device
\$D8 / \$D3 / \$BB	copy file / random number / GUID
\$D2	get time (7 bytes)
\$EB / \$E3	set baud rate / high-speed index
\$C8 \$C7 \$C6 \$C5 \$C2	hash: input, compute, length, output, clear
\$D0 \$CF \$CE \$CD	Base64 encode: input, compute, length, output
\$CC \$CB \$CA \$C9	Base64 decode: input, compute, length, output
\$BC \$BD \$BE \$BF	QR: input, encode, length, output



APPENDIX C

THE NDEV HANDLER

The complete source of the N: handler, verbatim from the `fujinet-nhandler` repository (`handler/src/ndev.s`), assembled with MADS. It is a CIO handler installed into HATABS as device **N**, with the six standard vectors — OPEN, CLOSE, GET, PUT, STATUS, SPECIAL — each built on a shared `DOSIOV` that copies a DCB template into page three and calls `SI0V`. Read it against Chapters 3 and 4: the `SPECIAL` routine's local handling of command 15 (flush), the inquiry (`$FF`) before every other special, the PUT buffer that flushes on EOL or when full, and the PROCEED interrupt that drives GET.

```
;; N: Device Handler
;; Compile with MADS

;; Author: Thomas Cherryhomes
;; <thom.cherryhomes@gmail.com>

;; CURRENT IOCB IN ZERO PAGE

DEBUG = 0 ; displays debug messages
DONT_RELOCATE = 0 ; set to one if you don't want to relocate code.

ZIOCB = $20 ; ZP IOCB
ZICHID = ZIOCB ; ID
ZICDNO = ZIOCB+1 ; UNIT #
ZICCOM = ZIOCB+2 ; COMMAND
ZICSTA = ZIOCB+3 ; STATUS
ZICBAL = ZIOCB+4 ; BUF ADR LOW
ZICBAH = ZIOCB+5 ; BUF ADR HIGH
ZICPTL = ZIOCB+6 ; PUT ADDR L
ZICPTH = ZIOCB+7 ; PUT ADDR H
ZICBLL = ZIOCB+8 ; BUF LEN LOW
ZICBLH = ZIOCB+9 ; BUF LEN HIGH
ZICAX1 = ZIOCB+10 ; AUX 1
ZICAX2 = ZIOCB+11 ; AUX 2
ZICAX3 = ZIOCB+12 ; AUX 3
ZICAX4 = ZIOCB+13 ; AUX 4
ZICAX5 = ZIOCB+14 ; AUX 5
ZICAX6 = ZIOCB+15 ; AUX 6

DOSINI = $0C ; DOSINI

VPRCED = $0202 ; PROCEED VCTR
MEMLO = $02E7 ; MEM LO
DVSTAT = $02EA ; 4 BYTE STATS

; PAGE 3
; DEVICE CONTROL BLOCK (DCB)

DCB = $0300 ; BASE
DDEVIC = DCB ; DEVICE #
DUNIT = DCB+1 ; UNIT #
DCOMND = DCB+2 ; COMMAND
DSTATS = DCB+3 ; STATUS/DIR
DBUFL = DCB+4 ; BUF ADR L
DBUFH = DCB+5 ; BUF ADR H
DTIMLO = DCB+6 ; TIMEOUT (S)
DRSVD = DCB+7 ; NOT USED
DBYTL = DCB+8 ; BUF LEN L
DBYTH = DCB+9 ; BUF LEN H
DAUXL = DCB+10 ; AUX BYTE L
DAUXH = DCB+11 ; AUX BYTE H

HATABS = $031A ; HANDLER TBL

; IOCB'S * 8

IOCB = $0340 ; IOCB BASE
ICHID = IOCB ; ID
ICDNO = IOCB+1 ; UNIT #
ICCOM = IOCB+2 ; COMMAND
```

```

ICSTA = IOCB+3 ; STATUS
ICBAL = IOCB+4 ; BUF ADR LOW
ICBAH = IOCB+5 ; BUF ADR HIGH
ICPTL = IOCB+6 ; PUT ADDR L
ICPTH = IOCB+7 ; PUT ADDR H
ICBLT = IOCB+8 ; BUF LEN LOW
ICBLH = IOCB+9 ; BUF LEN HIGH
ICAX1 = IOCB+10 ; AUX 1
ICAX2 = IOCB+11 ; AUX 2
ICAX3 = IOCB+12 ; AUX 3
ICAX4 = IOCB+13 ; AUX 4
ICAX5 = IOCB+14 ; AUX 5
ICAX6 = IOCB+15 ; AUX 6

; HARDWARE REGISTERS

PACTL = $D302 ; PIA CTRL A

; OS ROM VECTORS

CIOV = $E456 ; CIO ENTRY
SIOV = $E459 ; SIO ENTRY

; CONSTANTS

DEVIDN = $71 ; SIO DEVID
DSREAD = $40 ; FUJI->ATARI
DSWRIT = $80 ; ATARI->FUJI
MAXDEV = 8 ; # OF N: DEVS
EOF = $88 ; ERROR 136
EOL = $9B ; EOL CHAR

;; ORG HERE
ORG $6000

RELOCATE_CODE_START:
;; This is for OS/A+ ;;;;;;;;;;;;;;;;;;;;;;;;;;

RTS ; Immediately exit

;;; RESET HANDLER ;;;;;;;;;;;;;;;;;;;;;;;;;;

RESET:
JSR $FFFF ; pointing to original DOSINI
LDA RELOC_HANDLEREND ; Driver end LO
STA MEMLO
LDA RELOC_HANDLEREND+1 ; Driver end HI
STA MEMLO+1
JSR IHTBS ; Insert into HATABS
JSR CLALL
RTS

;;; END RESET HANDLER ;;;;;;;;;;;;;;;;;;;;;;;;;;

;;; INTERRUPT HANDLER ;;;;;;;;;;;;;;;;;;;;;;;;;;

INTR: LDA #$01 ; set trip to 1
STA TRIP
PLA
RTI

;;; END INTERRUPT HANDLER ;;;;;;;;;;;;;;;;;;;;;;;;;;

;;; SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;

;; GET IOCB UNIT # INTO X

GDIDX: LDX ZICDNO ; CURRENT IOCB UNIT #
DEX ; -1
RTS

;; Poll for Status

POLL: LDA ZICDNO ; Get Unit #
STA POLDCB+1 ; Put into Table
LDA RELOC_POLDCB ; Set up STATUS POLL DCB table
LDY RELOC_POLDCB+1

```

```

JSR DOSIOV      ; And do SIOV, fall thru to...

;; Update Bytes waiting

GETBW: JSR GDIDX      ; Get Unit #
      TXA              ; move to A for ASL
      ASL              ; * 2
      LDA DVSTAT      ; Get Bytes Waiting (LO)
      STA BW,X         ; Store in BW,(unit)
      LDA DVSTAT+1    ; Get Bytes Waiting (HI)
      STA BW+1,X      ; Store in BW,(unit)+1

      RTS

;; Save DVSTAT values

SVSTAT: JSR GDIDX      ; Get Unit into X
      JSR CAPRX        ; Cap RX values
      LDA DVSTAT      ; Get RX bytes waiting
      STA RLEN,X       ; Save RX bytes waiting
      LDA DVSTAT+2    ; Get Server Client connected/disconnected?
      STA DVS2,X       ; Save
      LDA DVSTAT+3    ; Get last error
      STA DVS3,X       ; Save
      RTS

;; Enable PROCEED interrupt

ENPRCD: LDA PACTL      ; Get PACTL register
      ORA #$01        ; Enable PROCEED
      STA PACTL       ; Store it back
      RTS

;; Disable PROCEED interrupt

DIPRCD: LDA PACTL      ; Get PACTL register
      AND #$FE        ; Disable PROCEED
      STA PACTL       ; store it back.
      RTS

;; Flush TX Buffer out

FLUSH: JSR GDIDX      ; UNIT NUMBER into X
      LDA ZICDNO      ; IOCB UNIT #
      STA FLUDCB+1    ; Put into table.
      LDA TOFF,X       ; get Transmit offset (# of bytes to send)
      BEQ FLDONE       ; Don't do anything if TX cursor is at 0.
      STA FLUDCB+8    ; Put into Table (Len and Aux)
      STA FLUDCB+10
      LDA RELOC_FLUDCB ; Copy Table to DCB
      LDY RELOC_FLUDCB+1
      JSR DOSIOV      ; And call SIOV
      JSR GDIDX      ; Get Unit into X
      LDA #$00        ; Clear TOFF
      STA TOFF,X
      LDY DSTATS
FLDONE: RTS           ; Done, LDY has DSTATS

;; Cap RX to 127 bytes (temporary routine)

CAPRX: LDA DVSTAT+1    ; Get hi-byte
      BNE CAPADJ      ; Adjust if > 256 bytes
      LDA DVSTAT      ; Get lo-byte
      BPL CAPDON      ; Exit if < 127 bytes
CAPADJ: LDA #$7F       ; 127 bytes
      STA DVSTAT      ; into DVSTAT/DVSTAT+1
      LDA #$00
      STA DVSTAT+1
CAPDON: RTS           ; Done

;; Close all IOCBs

CLALL: LDA #MAXDEV     ; Close all 4 N: devices
      STA TRIP        ; Temporarily use trip
CLLP: LDA TRIP         ; Get

```

```

STA ZICDNO      ; Store into unit #
JSR CLOSE      ; Close Nx:
DEC TRIP        ; Decrement
LDA TRIP        ; Get it
BNE CLLP        ; Loop until done.
RTS

;; Do read from ZIOCB unit

READ: JSR GDIDX      ; unit into X
LDA #$00        ; Set 0 into
STA ROFF,X      ; RXD cursor.
LDA ZICDNO      ; Get Unit #
STA READCB+1     ; Put into Read DCB table
LDA RLEN,X      ; Get RLEN (from status)
BEQ RDONE       ; If RLEN=0 then abort read.
STA READCB+8     ; Store in DBYTL
STA READCB+10    ; Store in DAUX1
LDA RELOC_READCB ; Set up Read DCB
LDY RELOC_READCB+1 ; ...
JSR DOSIOV      ; Do SIO call
LDY DSTATS      ; Get DSTATS for error
CPY #144        ; Is it 144?
BNE RDONE       ; No, simply return DSTATS in Y
JSR POLL        ; Otherwise, do a poll to get extended error
LDY DVSTAT+3     ; And return it in Y.
RDONE: RTS      ; Done.

;;; HANDLER RUNAD HERE ;;;;;;;;;;;;;;

START: LDA DOSINI      ; store the old dosini in our routine
STA RESET+1
LDA DOSINI+1
STA RESET+2

LDA RELOC_RESET ; add our reset handler
STA DOSINI
LDA RELOC_RESET+1
STA DOSINI+1

LDA RELOC_HANDLEREND ; update memlo
STA MEMLO
LDA RELOC_HANDLEREND+1
STA MEMLO+1
JSR CLALL      ; Close all

;;; Insert Handler entry into HATABS ;;;;;;;;;;

IHTBS: LDY #$00      ; Start at beginning of HATABS
IH1: LDA HATABS,Y
BEQ HFND        ; Did we find a blank ($00) entry?
CMP #'N'        ; or did we find our existing 'N' entry?
BEQ HFND        ; If so, insert our entry here.
INY            ; Otherwise, scoot forward to next entry.
INY
INY
CPY #11*3       ; Are we at the end of the table?
BCC IH1         ; Check again.

;; We found a slot, insert it in.

HFND: LDA #'N'      ; We are the N: device
STA HATABS,Y      ; first byte in our entry
LDA RELOC_DEVHDL   ; Get address of our handler table
STA HATABS+1,Y    ; and put it in Hatabs
LDA RELOC_DEVHDL+1
STA HATABS+2,Y

;; And vector in PROCEED.

VPRCD: LDA RELOC_INTR ; Get Addr of interrupt handler
STA VPRCED        ; Store it in PROCEED vector
LDA RELOC_INTR+1
STA VPRCED+1

;; We're done, back to DOS.

```

```

RTS

;;; INDICATE SUCCESS ;;;;;;;;;;;;;;;;;;;;;;;;;;

SUCC: LDY #$01      ; Indicate success
      RTS          ; Back to caller.

;;; CLEAR BUFFERS FOR UNIT X ;;;;;;;;;;;;;;;;;

CLRBUF: LDA #$00
        STA RLEN,X
        STA TOFF,X
        STA ROFF,X
        STA DVS2,X
        STA DVS3,X
        RTS

;;; COPY TABLE TO DCB AND DO SIO CALL ;;;;;;;;;;

DOSIOV: STA DODCBL+1 ; Set source address
        STY DODCBL+2
        LDY #$0B     ; 12 bytes
DODCBL: LDA $FFFF,Y  ; Changed above.
        STA DCB,Y    ; To DCB table
        DEY          ; Count down
        BPL DODCBL   ; Until done

SIOVDST:
        JSR SIOV     ; Call SIOV
        LDY DSTATS   ; Get STATUS in Y
        TYA          ; Copy it into A
        RTS          ; Done

;;; OPEN ;;;;;;;;;;;;;;;;;;;;;;;;;;

;; Fill in the OPEN table

OPEN:   JSR GDIDX    ; Set IOCB OFFSET TO UNIT #
        JSR CLRBUF   ; Clear Buffers
        LDA ZICDNO   ; GET Desired unit #
        STA OPNDCB+1 ; Store in open table
        LDA ZICBAL   ; Get desired buffer L0
        STA OPNDCB+4 ; Store in open table
        LDA ZICBAH   ; Get desired buffer HI
        STA OPNDCB+5 ; Store in open table
        LDA ZICAX1   ; Get requested Aux1
        STA OPNDCB+10 ; Store in open table
        LDA ZICAX2   ; Get requested Aux2
        STA OPNDCB+11 ; Store in open table

;; Do the SIOV call

        LDA RELOC_OPNDCB
        LDY RELOC_OPNDCB+1
        JSR DOSIOV

;; Return DSTATS in Y, unless 144, then get ext err.

        CPY #144     ; Did we get an ERROR- 144?
        BNE OPDONE   ; Nope, keep DSTATS in Y

;; We got a 144, get error from STATUS call
        JSR POLL     ; Do Status poll
        LDY DVSTAT+3 ; Get error code

OPDONE: RTS

;;; CLOSE ;;;;;;;;;;;;;;;;;;;;;;;;;;

CLOSE: JSR DIPRCD    ; Disable PROCEED
        JSR FLUSH    ; do PUT flush if needed.
        JSR CLRBUF   ; Clear buffer pointers
        LDA ZICDNO   ; Unit #
        STA CLODCB+1 ; Put into table

```

```

LDA RELOC_CLODCB ; Close DCB table
LDY RELOC_CLODCB+1
JSR DOSIOV ; Do SIOV

CLV
BVC SUCC ; Always return success

;;; GET ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

GET: JSR GDIDX ; Unit into X
LDA RLEN,X ; Get current RX len from last STATUS
BNE GETDRN ; If RLEN > 0 then drain.

;; Otherwise, we wait for something to happen.

GETWAI: JSR ENPRCD ; Enable Proceed
LDA TRIP ; Did trip change?
BEQ GETWAI ; Nope, not yet...

;; Something happened, try to poll for data.

JSR POLL ; Do Status Poll
JSR SVSTAT ; Save Status
JSR READ ; Do read

;; If RLEN=0 then determine if error.

LDA DVSTAT ; Get RLEN Again
BNE GETDRN ; If RLEN > 1, then drain.
LDY DVSTAT+3 ; Get ext err
CPY #136 ; EOF?
BEQ GETDNE ; Yes, return it.
LDY DSTATS ; Else, get DSTATS from status/read.
CPY #144 ; is it 144?
BNE GETDNE ; Nope, simply return it in Y, done.
LDY DVSTAT+3 ; Get Extended error
BNE GETDNE ; Done.

;; Drain

GETDRN: JSR DIPRCD ; Disable PROCEED
JSR GDIDX ; Get Unit into X again
DEC RLEN,X ; Decrement length
LDY ROFF,X ; Get Current Offset into X
LDA RBUF,Y ; Get next character
INC ROFF,X ; Increment cursor
TAY ; Store in Y for a moment

;; If RX buffer empty, turn off trip.

LDA RLEN,X ; Get RLEN
BNE GETDN2 ; some left, just go done with success
STA TRIP ; Otherwise store 0 into trip

GETDN2: TYA ; Bring back char into A
LDY #$01 ;
GETDNE: RTS

;;; PUT ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

PUT: JSR GDIDX ; Get Unit # into X
LDY TOFF,X ; Get TX cursor
STA TBUF,Y ; Put char into buffer ptd by cursor

INC TOFF,X ; Increment TX cursor

;; Do a FLUSH if EOL or buffer full

CMP #EOL ; EOL?
BEQ PFLUSH ; Do flush
CPY #$7F ; At end of buffer?
BNE PUTDON ; Nope, done.
PFLUSH: JSR FLUSH ; Do Flush.
PUTDON: RTS ; We're done.

```

```

;;; STATUS ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
STATUS: JSR ENPRCD      ; Enable PROCEED.

    ;; Return cached value if we still have data in RX

    JSR GDIDX      ; Unit into X
    LDA RLEN,X      ; Get RX len
    BNE STRETC      ; Return cached value if RLEN > 0

    LDA TRIP        ; Get TRIP?
    BEQ STRETC      ; No trip? Return cached.

    JSR POLL        ; RLEN = 0, do poll.
    JSR SVSTAT      ; Save DVSTAT values
    JSR READ        ; Do read.

STRETC: JSR      GDIDX      ; Unit into X
        LDA BW,X      ; Get Saved DVSTAT+0 val
    STA DVSTAT      ; Store into DVSTAT
    LDA BW+1,X      ; Get Saved DVSTAT+1 val
    STA DVSTAT+1    ; Store into DVSTAT+1
    LDA DVS2,X      ; Get Saved DVSTAT+2 val
    STA DVSTAT+2    ; Store
    LDA DVS3,X      ; Get Saved DVSTAT+3 val
    STA DVSTAT+3
    TAY              ; copy it into Y for error output.

    RTS              ; Done.

;;; SPECIAL ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
SPECIAL:

    ;; Clear Trip

    LDA #$00
    STA TRIP

    ;; Handle Local Commands

    LDA ZICCOM
    CMP #$0F        ; 15 = FLUSH
    BNE SPQ          ; No. Handle protocol commands
    JSR FLUSH        ; Yes. Do flush.
    LDY #$01        ; Flush always successful
    JMP SPCDNE        ; We're done.

    ;; Handle Protocol commands, do INQDS Query

SPQ:  LDA ZICDNO      ; Get Unit #
    STA SPQDCB+1      ; Store in table
    LDA ZICCOM        ; Get Command
    STA SPQDCB+10     ; Put in AUX1 for query
    LDA RELOC_SPQDCB  ; Set up SPECIAL QUERY DCB TABLE
    LDY RELOC_SPQDCB+1 ;
    JSR DOSIOV        ; Do Query
    LDY DSTATS        ; Get DSTATS
    BMI SPCDNE        ; SIO error, return in Y. There is no ext err.

    ;; We got a query, if it's $FF, return unimplemented.
    LDA INQDS         ; Get the Returned DSTATS value from inquiry
    CMP #$FF          ; Is it $FF ?
    BNE SPD0          ; Nope, let's do it.
    LDY #146          ; ERROR- 146 Unimplemented Command
    JMP SPCDNE        ; Done.

    ;; Do the Special, get all IOCB params, push onto stack

SPD0: STA SPEDCB+3    ; DSTATS value from inquiry
    LDA ZICDNO        ; Unit #
    STA SPEDCB+1      ; Command
    LDA ZICCOM        ; Command
    STA SPEDCB+2
    LDA ZICBAL        ; Ptr to passed in devicespec
    STA SPEDCB+4
    LDA ZICBAH        ;

```

```

STA SPEDCB+5
LDA ZICAX1      ; Aux1
STA SPEDCB+10
LDA ZICAX2      ; Aux2
STA SPEDCB+11
LDA RELOC_SPEDCB
LDY RELOC_SPEDCB+1
JSR DOSIOV

;; Get error and return extended if needed.

LDY DSTATS      ; Get DSTATS
CPY #144        ; Is it 144?
BNE SPCDNE      ; Nope, just return it.

JSR POLL        ; Get status, for error
LDY DVSTAT+3    ; Get extended error.

SPCDNE: RTS

RELOCATE_CODE_END:
; -----
; ---- DATA SECTION ----
; -----

READCB .BYTE DEVIDN      ; DDEVIC
       .BYTE $FF         ; DUNIT
       .BYTE 'R'         ; DCOMND
       .BYTE DSREAD      ; DSTATS
rel100 .WORD RBUF        ; DBUF
       .BYTE $1F         ; DTIMLO
       .BYTE $00         ; DRESVD
       .BYTE $FF         ; DBYTL
       .BYTE $00         ; DBYTH
       .BYTE $FF         ; DAUX1
       .BYTE $00         ; DAUX2

;;; END SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;;

FLUDCB: .BYTE DEVIDN      ; DDEVIC
       .BYTE $FF         ; DUNIT
       .BYTE 'W'         ; DCOMND
       .BYTE DSWRIT      ; DSTATS
rel101 .WORD TBUF        ; DBUFL
       .BYTE $1F         ; DTIMLO
       .BYTE $00         ; DRESVD
       .BYTE $FF         ; DBYTL
       .BYTE $00         ; DBYTH
       .BYTE $FF         ; DAUX1
       .BYTE $00         ; DAUX2

POLDCB: .BYTE DEVIDN      ; DDEVIC
       .BYTE $FF         ; DUNIT
       .BYTE 'S'         ; DCOMND
       .BYTE DSREAD      ; DSTATS
       .WORD DVSTAT      ; DBUF
       .BYTE $1F         ; DTIMLO
       .BYTE $00         ; DRESVD
       .WORD 4           ; 4 bytes
       .BYTE $00         ; DAUX1
       .BYTE $00         ; DAUX2

;;; DEVICE HANDLER TABLE ;;;;;;;;;;;;;;;;;;;;;;;;;;;

DEVHDL:
rel110 .WORD OPEN-1
rel111 .WORD CLOSE-1
rel112 .WORD GET-1
rel113 .WORD PUT-1
rel114 .WORD STATUS-1
rel115 .WORD SPECIAL-1

;; OPEN DCB TABLE

OPNDCB:

```



```

.BYTE DEVIDN      ; DDEVIC
.BYTE $FF        ; DUNIT
.BYTE '0'         ; DCOMND
.BYTE DSWRIT      ; DSTATS
.BYTE $FF        ; DBUFL
.BYTE $FF        ; DBUFH
.BYTE $1F        ; DTIMLO
.BYTE $00        ; DRESVD
.BYTE $00        ; DBYTL
.BYTE $01        ; DBYTH
.BYTE $FF        ; DAUX1
.BYTE $FF        ; DAUX2

CLODCB .BYTE DEVIDN      ; DDEVIC
.BYTE $FF        ; DUNIT
.BYTE 'C'         ; DCOMND
.BYTE $00        ; DSTATS
.BYTE $00        ; DBUFL
.BYTE $00        ; DBUFH
.BYTE $1F        ; DTIMLO
.BYTE $00        ; DRESVD
.BYTE $00        ; DBYTL
.BYTE $00        ; DBYTH
.BYTE $00        ; DAUX1
.BYTE $00        ; DAUX2

SPQDCB .BYTE DEVIDN      ; DDEVIC
.BYTE $FF        ; DUNIT
.BYTE $FF        ; DCOMND ; inq
.BYTE DSREAD      ; DSTATS
rel120 .WORD INQDS      ; DBUFL, DBUFH
.BYTE $1F        ; DTIMLO
.BYTE $00        ; DRESVD
.BYTE $01        ; DBYTL ; 1 byte
.BYTE $00        ; DBYTH
.BYTE $FF        ; DAUX1
.BYTE $00        ; DAUX2

SPEDCB .BYTE DEVIDN      ; DDEVIC
.BYTE $FF        ; DUNIT
.BYTE $FF        ; DCOMND ; special cmd
.BYTE $FF        ; DSTATS ; DSTATS from inquiry
.BYTE $00        ; DBUFL
.BYTE $00        ; DBUFH
.BYTE $1F        ; DTIMLO
.BYTE $00        ; DRESVD
.BYTE $00        ; DBYTL ; 256 bytes
.BYTE $01        ; DBYTH
.BYTE $FF        ; DAUX1
.BYTE $FF        ; DAUX2

;; End of Handler

;;; VARIABLES ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

TRIP .ds 1      ; Interrupt Tripped?
RLEN .ds MAXDEV ; RXD Len
ROFF .ds MAXDEV ; RXD offset cursor
TOFF .ds MAXDEV ; TXD offset cursor
INQDS .ds 1     ; DSTATS to return in inquiry
DVS2 .ds MAXDEV ; DVSTAT+2 SAVE
DVS3 .ds MAXDEV ; DVSTAT+3 SAVE

BW      .ds      MAXDEV * 2      ; Bytes waiting save for status.

RBUF .ds 128    ; RXD buffer
TBUF .ds 128    ; TXD buffer

; LDA #<ADDR LO becomes LDA RELOC_ADDR
; LDY #>ADDR HI becomes LDA RELOC_ADDR+1

relocate_000
RELOC_OPNDCB .WORD OPNDCB

relocate_001

```

```

RELOC_READCB .WORD READCB

relocate_002
RELOC_POLDCB .WORD POLDCB

relocate_003
RELOC_FLUDCB .WORD FLUDCB

relocate_004
RELOC_RESET .WORD RESET

relocate_005
RELOC_HANDLEREND .WORD HANDLEREND

relocate_006
RELOC_DEVHDL .WORD DEVHDL

relocate_007
RELOC_INTR .WORD INTR

relocate_008
RELOC_CLODCB .WORD CLODCB

relocate_009
RELOC_SPQDCB .WORD SPQDCB

relocate_010
RELOC_SPEDCB .WORD SPEDCB

NEW_START .BYTE $4C
relocate_011 .WORD START

RELOCATE_DATA_END:

HANDLEREND = *

; Within your code you need the following tables
; in this order
;
; RELOCATE_CODE_START
; <code>
; RELOCATE_CODE_END
; <data>
; reloc000 .WORD <address>
; RELOCATE_DATA_END
;
; RELOCATION_TABLE
; .WORD <location of addresses that need relocating>
; .WORD 0 ; END OF TABLE

RELOCATION_TABLE:
.WORD relocate_000,relocate_001,relocate_002,relocate_003,relocate_004
.WORD relocate_005,relocate_006,relocate_007,relocate_008,relocate_009
.WORD relocate_010, relocate_011
.WORD rel100,rel101
.WORD rel110,rel111,rel112,rel113,rel114,rel115
.WORD rel120
; icl "CIO-Routines_RELOC.ASM"
; Code relocation data will be auto-magically appended to the end of the table

.WORD 0 ; end of table
.DS 1024 ; this is only needed now for testing. This storage won't be necessary in a future release
END_RELOCATION_TABLE:

icl "RELOCATE.ASM"

.if DEBUG = 1

; This debug code simply displays
; if it's relocated or not and
; where it is loaded/relocated

icl "CIO-Routines.asm"
icl "CIO-Routines_DATA.ASM"

MSG .BYTE 'Relocated to ',EOL

```

```

NMSG .BYTE 'Not relocated ',EOL
PRGNAME .BYTE 'NDEV: 2021-01-05 18:45',EOL
.endif

RELOCATE_START

.if DEBUG = 1

    JSR OPEN_E_DEVICE
    LDA #<PRGNAME
    LDY #>PRGNAME
    JSR PRINT_STRING

.if DONT_RELOCATE = 0
    LDA #<MSG
    LDY #>MSG
.else
    LDA #<NMSG
    LDY #>NMSG
.endif ; DONT_RELOCATE

    JSR PRINT_STRING_NO_EOL

.if DONT_RELOCATE = 0
    LDA MEMLO+1
.else
    LDA #>RELOCATE_CODE_START
.endif ; DONT_RELOCATE

    JSR PRINT_HEX_NO_EOL

.if DONT_RELOCATE = 0
    LDA MEMLO
.else
    LDA #<RELOCATE_CODE_START
.endif ; DONT_RELOCATE

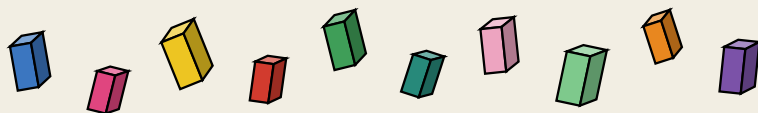
    JSR PRINT_HEX
    LDA #<NORE
    LDY #>NORE
    JSR PRINT_STRING

.endif ; DEBUG

.if DONT_RELOCATE = 0
    JSR RELOCATE_TO_MEMLO
.endif ; DONT_RELOCATE
JMP NEW_START

ORG $02E2 ; init
.WORD RELOCATE_START
END

```



APPENDIX D

NETCAT

Every guide in this series ends with a **netcat** — open a socket, pump bytes both ways — because it exercises the whole road in a page. Here it is on the **high road**, in Atari BASIC through **N:**: whatever you type crosses the wire, whatever arrives paints on the screen, until the far end hangs up or you press **BREAK**.

LISTING 20. NETCAT, IN ATARI BASIC OVER N:

```
10 REM ===== FUJINET NETCAT (N: HANDLER) =====
20 DIM L$(255)
30 REM OPEN A RAW TCP SOCKET, READ/WRITE, BINARY
40 OPEN #1,12,0,"N1:TCP://192.168.1.5:9000/"
45 OPEN #2,4,0,"K:":REM THE KEYBOARD, FOR TYPING
50 REM --- MAIN PUMP ---
60 STATUS #1,S
70 IF PEEK(748)=0 THEN GOTO 900:REM FAR END CLOSED
80 IF PEEK(749)=136 THEN GOTO 900:REM EOF
90 BW=PEEK(746)+PEEK(747)*256
100 IF BW=0 THEN GOTO 200
110 REM --- DRAIN THE SOCKET TO THE SCREEN ---
120 FOR I=1 TO BW
130 GET #1,B:PUT #16,B:REM #16 = SCREEN (SEE BELOW)
140 NEXT I
200 REM --- SEND A KEY IF ONE IS WAITING ---
210 IF PEEK(764)=255 THEN GOTO 60:REM NO KEY DOWN
220 GET #2,K:REM READ THE ATASCII KEY
230 PUT #1,K
240 XIO 15,#1,0,0,"N:":REM FLUSH IT OUT NOW
250 GOTO 60
900 PRINT :PRINT "*** CONNECTION CLOSED"
910 CLOSE #1:CLOSE #2:END
```

Three Atari touches earn their keep. **PEEK(764)** is the OS's "last key pressed" cell — **255** when nothing is down — so the pump never blocks waiting for the keyboard while the socket has data. **XIO 15** flushes each keystroke the instant you type it, so the far end sees your typing live instead of a line at a time. And **PUT #16** is the old Atari BASIC trick for writing to the **screen editor**: BASIC multiplies the channel number by sixteen to index the IOCB, so channel 16 wraps to IOCB 0 — the **E:** editor BASIC reserves for itself and will not let you **PUT #0**. **PUT #16,B** slips a byte onto the screen through that back door, faster than **PRINT CHR\$(B)**.

On the other end, anything that speaks TCP will do — the classic test is the Unix **nc** itself, listening on the port you named:

```
nc -l 9000
HELLO FROM THE ATARI
and hello back from the laptop
the quick brown fox jumped over
** CONNECTION CLOSED
```

Type, and your keystrokes cross the room — or the world — and the reply paints onto a 1.79 MHz machine that predates the network it just joined. Fill the DCB, call the OS, read the reply; the rest is just 6502.

SOURCE-VERIFIED. Every command byte, parameter and payload in this book was read out of the FujiNet sources, not remembered: **fujinet-firmware** — the SIO bus in **lib/bus/sio/**, the device handlers in **lib/device/sio/** (**network.cpp**, **sioFuji.cpp**) and **lib/device/fujiDevice.cpp**, the command list in **include/fujiCommandID.h** and device ids in **include/fujiDeviceID.h**; the N: handler **fujinet-nhandler** (**handler/src/ndev.s**, reproduced in Appendix C); and the Action! library **NIO.ACT**. CIO and SIO facts are drawn from the Atari 400/800 OS ROM source listing.

This guide is free software, part of the **fujinet-manuals** repository, released under the GNU General Public License v3. The FujiNet community answers day and night on Discord.